Stato dell'Arte nella Stima dei Costi di Progetti Software

Fabrizio Apuzzo Agostino Cesarano

August 4, 2026

1 Abstract

Il presente documento ha lo scopo di fornire una panoramica dello stato dell'arte nella stima dei costi di realizzazione di progetti software, esplorando e confrontando nel dettaglio due soluzioni recenti. L'articolo di Di Martino et. al.[2] valuta empiricamente l'efficacia di cinque metodi approssimati di misurazione delle dimensioni funzionali (Functional Size Measurement, FSM) come input per predittori lineari dello sforzo di sviluppo: due approssimazioni della Function Point Analysis (FPA) proposte da IFPUG (High Level FPA, Indicative FPA), e tre approssimazioni del metodo COSMIC proposte dalla COSMIC Organization (Average Functional Process, Fixed Size Classification, Equal Size Bands). L'articolo [1] propone invece un modello di deep learning, denominato F-DCNN, che combina una fase di clusterizzazione fuzzy con una rete neurale convoluzionale profonda per la stima dello sforzo, confrontandolo con diverse tecniche di machine learning tradizionali.

2 Introduzione

La stima dei costi di progetti software è un'attività critica per la pianificazione e la gestione dei progetti: il successo di un progetto dipende in larga misura dalla capacità di stimare accuratamente i costi e le risorse necessarie. Tuttavia, la natura imprevedibile e complessa del processo di sviluppo software rende questa attività particolarmente difficile. Negli ultimi decenni, sono stati sviluppati numerosi approcci per affrontare questa sfida, ciascuno con i propri punti di forza e limitazioni. In questo documento, esploreremo lo stato dell'arte nella stima dei costi di progetti software, concentrandoci su due approcci specifici: uno basato sull'approssimazione di misure funzionali classiche (FPA e COSMIC), l'altro su una rete neurale convoluzionale guidata da clusterizzazione fuzzy.

3 Lo Stato dell'Arte

L'articolo [6] identifica e analizza quattro principali macro-approcci nello stato dell'arte per la stima dei costi di progetti software: stime basate su algoritmi, stime basate sul giudizio di esperti, stime basate su modelli di apprendimento automatico, metodi ibridi.

3.1 Stime Basate su Algoritmi

I metodi basati su algoritmi sono approcci che utilizzano formule matematiche e modelli statistici per stimare lo sforzo in funzione di variabili come la dimensione e complessità del software da realizzare e le caratteristiche del team di sviluppo. L'articolo di riferimento si riferisce esclusivamente a metodi di stima dell'effort: la stima dell'effort e stima della dimensione, per quanto strettamente correlati, sono due concetti distinti.

- **Metodi di stima dell'effort** (ad esempio COCOMO e SLIM), che calcolano lo sforzo di sviluppo a partire da una misura della dimensione del software già disponibile (ad esempio, espressa in linee di codice o in Function Point), combinandola con altri cost driver (esperienza del team, complessità del progetto, vincoli tecnologici, ecc.);
- **Metodi di stima della dimensione**, o Functional Size Measurement (FSM, ad esempio Function Point Analysis e COSMIC), il cui scopo non è stimare direttamente lo sforzo, ma produrre la misura della dimensione che i metodi della prima categoria assumono come dato di input già noto.

Questa distinzione è rilevante perché i due gruppi di metodi hanno profili di costo ed economicità differenti. I metodi di stima dell'effort, una volta noti i valori dei cost driver, applicano una formula matematica in modo relativamente rapido ed economico, senza richiedere necessariamente l'intervento di un esperto nella fase di calcolo della stima finale [3]. I metodi di stima della dimensione, invece, richiedono tipicamente l'intervento di misuratori esperti e certificati: nel caso di FPA, ad esempio, la caratterizzazione delle componenti del sistema (la distinzione tra file logici interni ed esterni, input, output e interrogazioni, e la determinazione della loro complessità in termini di Data Element Types e Record Element Types) è un compito non banale, che richiede tempo ed esperienza specifica [2]. Non a caso, gli studi sulla produttività dei misuratori FPA riportano ritmi relativamente lenti, tra le 200 e le 600 Function Point al giorno [2], un fattore che ha motivato lo sviluppo di metodi di approssimazione come quelli discussi nella Sezione 4.

Al di là di questa distinzione interna, l'efficacia di entrambe le categorie di modelli algoritmici è limitata dalla difficoltà di adattarsi a contesti diversi da quelli per cui sono stati progettati o tarati, con conseguente bisogno costante di aggiornarne i parametri o le definizioni stesse per riflettere le nuove tecnologie e metodologie di sviluppo. La letteratura [5] evidenzia criticità nell'impiego del modello COCOMO II (già evoluzione di COCOMO I) a seguito della crescente

diffusione di novità come i metodi Agile o i servizi web e cloud. Una nuova versione del modello, denominata COCOMO III, veniva infatti già discussa anni fa pur non essendo ancora stata ufficialmente rilasciata.

3.2 Stime Basate sul Giudizio di Esperti

Gli approcci basati sul giudizio di esperti si fondano sull'esperienza e la conoscenza di professionisti del settore per stimare i costi di progetti software. Questi metodi possono essere soggettivi e influenzati da bias cognitivi e ambiguità, ma la letteratura mostra come l'esperienza e la competenza degli esperti possano portare a stime accurate; il paper [4] evidenzia che 10 sui 16 studi analizzati mostrano un vantaggio in termini di accuratezza delle stime basate sul giudizio di esperti rispetto a quelle basate su metodi automatici. Alcuni dei metodi più comuni includono:

- **Planning Poker** - diversi esperti stimano in parallelo lo sforzo necessario per completare un'attività, confrontando poi le stime e discutendo eventuali discrepanze fino a raggiungere un consenso.
- **Analisi degli Story Point** - gli esperti stimano lo sforzo necessario per completare un'attività in termini di Story Point, che rappresentano una misura relativa della complessità e del tempo richiesto per completare l'attività.
- **Fuzzy Set** - gli esperti utilizzano insiemi fuzzy per rappresentare l'incertezza e la vaghezza nelle stime per ogni componente del progetto, consentendo una maggiore trasparenza nella valutazione dello sforzo richiesto.

Questi metodi non sono formalmente definiti come i modelli algoritmici, e la loro concreta applicazione può variare notevolmente a seconda del contesto e delle preferenze degli esperti coinvolti. Tuttavia, la loro flessibilità, la loro capacità di evolversi, e la loro capacità di cogliere informazioni contestuali uniche e caratteristiche li rendono strumenti preziosi nella stima dei costi di progetti software.

3.3 Stime Basate su Modelli di Apprendimento Automatico

Gli approcci basati su machine learning sono potenzialmente più performanti rispetto ai metodi tradizionali, sono adattabili a contesti diversi e possono essere aggiornati con nuovi dati. Tuttavia, richiedono una quantità significativa di dati storici per addestrare i modelli e il loro processo decisionale non è facilmente interpretabile. Il costo operazionale di questi modelli può essere elevato, richiedendo risorse computazionali e competenze specialistiche per la loro implementazione e manutenzione. Gli approcci di machine learning più comuni includono:

- **Regressione Lineare** - un modello statistico che cerca di stabilire una relazione lineare tra le variabili indipendenti (ad esempio, la dimensione del software) e la variabile dipendente (ad esempio, lo sforzo stimato).

- **Alberi Decisionali** - un modello che utilizza una struttura ad albero per rappresentare le decisioni e le loro possibili conseguenze, basandosi su caratteristiche del progetto e del team di sviluppo.
- **Reti Neurali** - un modello ispirato al funzionamento del cervello umano, che utilizza strati di nodi interconnessi per apprendere relazioni complesse tra le variabili di input e output, come la dimensione del software e lo sforzo stimato.
- **Support Vector Machines (SVM)** - un modello che cerca di trovare un iperpiano ottimale per separare i dati in classi, basandosi su caratteristiche del progetto e del team di sviluppo.
- **Random Forest** - un modello che combina più alberi decisionali per migliorare la precisione delle stime, basandosi su caratteristiche del progetto e del team di sviluppo.
- **Gradient Boosting** - un modello che costruisce una serie di alberi decisionali in sequenza, dove ogni albero cerca di correggere gli errori del precedente, basandosi su caratteristiche del progetto e del team di sviluppo.

Tra quelli elencati, gli approcci basati su reti neurali sono una delle macro-categorie di modelli di machine learning più studiate e applicate nella stima dei costi di progetti software, anche se la loro complessità e la difficoltà di interpretazione dei risultati possono rappresentare una barriera significativa per l'adozione in contesti industriali. Le sezioni successive approfondiscono le caratteristiche e le sfide associate ad alcuni di questi approcci in particolare.

3.4 Approcci Ibridi

Gli approcci ibridi combinano elementi dei metodi precedenti per sfruttare i punti di forza di ciascun approccio. Ad esempio, un approccio ibrido potrebbe utilizzare un modello algoritmico per fornire una stima iniziale, che viene poi raffinata attraverso il giudizio di esperti. Si distinguono due principali categorie di approcci ibridi:

- **Omogenei** - si combina lo stesso modello di base con diverse configurazioni
- **Eterogenei** - si combinano modelli sostanzialmente diversi

Di loro natura, gli approcci ibridi sono più robusti e generalmente più accurati rispetto ai metodi non ibridi, ma possono essere più complessi da implementare e richiedere più risorse.

Queste macro-categorie sono difficili da definire in modo rigoroso e servono solo a fornire un quadro generale della letteratura. Il modello proposto da Di Martino et al. non può essere considerato un approccio ibrido, in quanto la prima fase di preprocessing dei dati (approssimazione della misura di dimensione) non è un modello di stima dello sforzo, ma un metodo per ottenere una

rappresentazione da fornire in input a un modello di regressione lineare semplice (SLR) per stimare lo sforzo. Quindi è considerabile un metodo basato su apprendimento automatico, in quanto utilizza un modello di regressione lineare semplice (SLR) per stimare lo sforzo di sviluppo. Per lo stesso motivo, il modello F-DCNN descritto nella Sezione 5, pur essendo scomponibile in due fasi sostanzialmete distinte (clustering fuzzy e rete neurale convoluzionale profonda), rappresenta anch'esso un metodo puramente di apprendimento automatico e non un approccio ibrido eterogeneo.

4 Confronto tra Due Approcci Recenti

L'articolo [2] affronta il problema della stima dei costi non proponendo un nuovo modello di stima, bensì valutando empiricamente l'efficacia di misure di dimensione funzionale approssimate come input per predittori lineari dello sforzo di sviluppo. Lo studio si concentra quindi sulla sotto-famiglia dei metodi di stima della dimensione (FSM), discussa nella Sezione 3.1, e lo fa definendo un metodo di stima dello sforzo basato su Regressione Lineare Semplice.

4.1 Metodo

Gli autori hanno raccolto dati su 25 progetti software industriali realizzati da un'unica azienda italiana, misurandone la dimensione con il metodo standard FPA, il metodo standard COSMIC, due approssimazioni di FPA proposte da IFPUG (High Level FPA e Indicative FPA), e tre approssimazioni di COSMIC proposte dalla COSMIC Organization (Average Functional Process, Fixed Size Classification, Equal Size Bands). Per ciascuna misura di dimensione è stato costruito un modello di Regressione Lineare Semplice per predire lo sforzo di sviluppo, espresso in ore-persona. Gli autori hanno deliberatamente evitato tecniche più sofisticate (machine learning, approcci search-based), poiché l'obiettivo dello studio era confrontare le misure di dimensione funzionale tra loro, non le tecniche di stima. La qualità dei modelli è stata valutata tramite cross-validation leave-one-out e la metrica Mean of Absolute Residuals (MAR), confrontando i risultati con due baseline costanti (media e mediana degli sforzi dei progetti precedenti) e verificando la significatività statistica delle differenze con test di Wilcoxon ed effect size di Vargha-Delaney.

4.2 Risultati

Per quanto riguarda le approssimazioni di FPA, il modello basato su High Level FPA (MAR = 561 ore-persona) supera significativamente le baseline (MAR rispettivamente 843 e 872 ore-persona) e ottiene un'accuratezza comparabile a quella del modello basato su FPA standard (MAR = 580), senza differenza statisticamente significativa. Al contrario, il modello basato su Indicative FPA (MAR = 760) non supera in modo significativo le baseline e risulta quindi

sconsigliato dagli autori, anche solo come stima approssimativa (Rough Order of Magnitude).

Per le approssimazioni di COSMIC, tutti e tre i modelli approssimati superano significativamente le baseline, con un effect size ampio. Confrontati con il modello COSMIC standard (MAR = 276), i modelli approssimati mostrano un errore via via minore al crescere della sofisticazione del metodo di approssimazione: Average Functional Process produce un errore circa il 40% superiore, Fixed Size Classification circa il 22% superiore, mentre Equal Size Bands (MAR = 295) mostra una differenza di circa il 7%, non statisticamente significativa rispetto al metodo standard.

4.3 Discussione

Gli autori concludono che le approssimazioni non sono equivalenti tra loro: High Level FPA ed Equal Size Bands rappresentano scelte valide quando il tempo o le informazioni disponibili sono limitati, mentre Indicative FPA, Average Functional Process e Fixed Size Classification introducono un degrado significativo dell'accuratezza (o, nel caso delle ultime due, richiedono comunque calibrazione su progetti pregressi e una classificazione soggettiva dei processi funzionali, con un tasso di errata classificazione osservato di poco superiore al 10%).

5 Stima dello Sforzo con Rete Convoluzionale e Clustering Fuzzy: Azzeh et al. (2024)

L'articolo [1] propone un modello denominato F-DCNN (Fuzzy Deep Convolutional Neural Network), che combina la logica fuzzy con una rete neurale convoluzionale profonda (DCNN) per la stima dello sforzo di sviluppo software. A differenza dell'approccio di Di Martino et al., che confronta diverse misure di dimensione mantenendo fissa la tecnica di stima (SLR), questo lavoro ha il solo scopo di proporre una nuova tecnica di stima, mantenendo come input le variabili numeriche già disponibili nei dataset di riferimento.

5.1 Motivazione e Metodo

Il punto di partenza degli autori è che le DCNN, pur ampiamente utilizzate in ambito computer vision, non erano mai state applicate alla stima dello sforzo software, poiché richiedono in input dati in forma di matrice bidimensionale, mentre i dataset di stima dello sforzo sono tipicamente vettoriali. Gli autori lo affermano esplicitamente:

> the DCNN was not used before for the problem of effort estimation because of the input mismatch. The DCNN requires input in the form of 2D matrix, which is not available on the current datasets[1].

Gli autori propongono una fase di feature engineering basata su Fuzzy C-Means (FCM): ogni dataset viene partizionato in k cluster, e per ciascuna

caratteristica (feature) del progetto viene costruita una funzione di appartenenza Gaussiana centrata sul centroide del cluster corrispondente. Il numero ottimale di cluster k viene selezionato tramite l'indice di validità Xie-Beni. Ne risulta, per ogni osservazione, una matrice di gradi di appartenenza (feature $\times$ cluster) che può essere fornita in input alla rete convoluzionale, analogamente a un'immagine. L'idea è che la rappresentazione fuzzy permetterebbe di catturare la natura del progetto software in maniera flessibile, consentendo al modello di catturare pattern più complessi rispetto a una rappresentazione vettoriale tradizionale: un progetto sarebbe idealmente collocabile in un cluster (quello dei progetti con tanti file di interfaccia e pochi logici), ma ha anche alcune caratteristiche che lo avvicinano a un altro cluster (quello dei progetti con pochi file di interfaccia e tanti logici), e quindi ha un grado di appartenenza non nullo anche a quest'ultimo, il modello apprende che modelli di questo tipo possono essere associati a un certo sforzo di sviluppo, e quindi può stimarlo in maniera più accurata. La rete F-DCNN è composta da due strati convoluzionali 2D con 16 filtri 3×3 ciascuno, due strati di max pooling, e uno strato completamente connesso con due livelli nascosti (32 e 16 neuroni), addestrata con ottimizzatore Adam e funzione di perdita basata sull'errore quadratico medio.

Il modello è stato valutato su alcuni dataset pubblici (Albrecht, CESAW, china, COCOMO-81, Desharnais, Finnish, Huijgens492, Kitchenham, Maxwell, Miyazaki94, Nasa93, renzo-pomodoro, Subbiah, UCP, Valdes-Souto) tramite cross-validation leave-one-out, confrontandolo con cinque tecniche di riferimento: Analogy-Based Effort Estimation (ABE), Support Vector Regression (SVR), Random Forest (RF), XGBoost e Multi-Layer Perceptron (MLP). Le metriche di valutazione utilizzate sono il Mean Absolute Error (MAE), la Standardized Accuracy (SA) e l'effect size Δ.

5.2 Risultati

(...)

5.3 Discussione e Limiti

Gli stessi autori segnalano alcune criticità del proprio approccio. In primo luogo, la scelta del numero di cluster k, per quanto guidata dall'indice Xie-Beni, ha un impatto rilevante sulla rappresentazione dei dati e meriterebbe un processo di ottimizzazione più approfondito. (...) In secondo luogo, l'uso dei gradi di appartenenza fuzzy al posto dei valori originali delle feature riduce l'interpretabilità del modello risultante, come dichiarato esplicitamente:

> we use membership values of instead of the actual value which might reduce the interpretability of the constructed model[1].

Infine, gli autori sottolineano che il metodo LOOCV utilizzato per la validazione, per quanto appropriato per dataset di piccole dimensioni, può risultare computazionalmente oneroso su dataset più ampi.

6 Confronto tra i Due Approcci e Considerazioni Conclusive

(...)

References

[1] Mohammad Azzeh, Abedalrhman Alkhateeb, and Ali Nassif. Software effort estimation using convolutional neural network and fuzzy clustering. *Neural Computing and Applications*, 36:14449–14464, 05 2024.

[2] Sergio Di Martino, Filomena Ferrucci, Carmine Gravino, and Federica Sarro. Assessing the effectiveness of approximate functional sizing approaches for effort estimation. *Information and Software Technology*, 123:106308, 2020.

[3] Wei Lin Du, Danny Ho, and Luiz Fernando Capretz. A neuro-fuzzy model with SEER-SEM for software effort estimation. *CoRR*, abs/1508.00032, 2015.

[4] Magne Jorgensen and Martin Shepperd. A systematic review of software development cost estimation studies. *IEEE Transactions on Software Engineering*, 33(1):33–53, 2007.

[5] Tim Menzies, Ye Yang, George Mathew, Barry W. Boehm, and Jairus Hihn. Negative results for software effort estimation. *CoRR*, abs/1609.05563, 2016.

[6] Kevin James Tomescu, Niamh Gowran, Lorena Gomez, Eoin Delahunty, Andrew McCarren, Gerard Marks, Murat Yilmaz, Richard Messnarz, and Paul M. Clarke. A review of estimation in software engineering. In Murat Yilmaz, Paul Clarke, Andreas Riel, Richard Messnarz, Mikus Zelmenis, and Ivi Anna Buce, editors, *Systems, Software and Services Process Improvement*, pages 158–175, Cham, 2026. Springer Nature Switzerland.